LAB # 09

NAIVE BAYES CLASSIFIER

Procedure:

Task 1:

Source Code And Output:



```
from sklearn.datasets import load_iris

data = load_iris()

X = data.data
y = data.target

print("Iris dataset loaded ")
print("features:", X.shape)
print("target:", y.shape)
print("feature names:", data.feature_names)
```

```
Iris dataset loaded
features: (150, 4)
target: (150,)
feature names: ['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)', 'petal width (cm)']
```

Task 2:

Source Code And Output:

```
[11]
✓ 0s
```

```python
from sklearn.datasets import load_iris
from sklearn.impute import SimpleImputer

data = load_iris()

X = data.data
y = data.target

imputer = SimpleImputer(strategy='mean')
X = imputer.fit_transform(X)

print("Missing values handled")
print("Preprocessed data:")
print(X[:5])
```

```
Missing values handled
Preprocessed data:
[[5.1 3.5 1.4 0.2]
 [4.9 3.  1.4 0.2]
 [4.7 3.2 1.3 0.2]
```

Task 3:

Source Code And Output:

```
0s
```

```python
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split

data = load_iris()

X = data.data
y = data.target

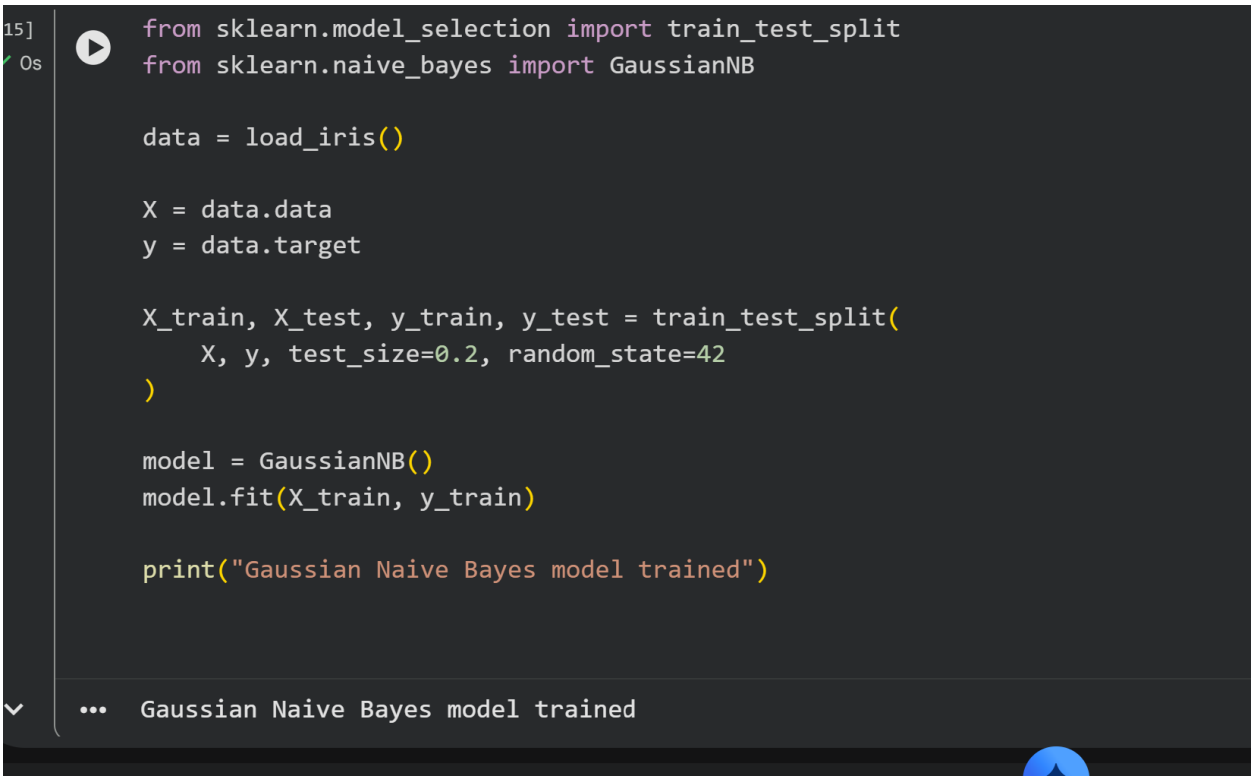
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

print("training:", X_train.shape)
print("testing:", X_test.shape)
```

```
training: (120, 4)
testing: (30, 4)
```

Task 4:

Source Code And Output:



```
15]
Os
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB

data = load_iris()

X = data.data
y = data.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

model = GaussianNB()
model.fit(X_train, y_train)

print("Gaussian Naive Bayes model trained")
```

```
Gaussian Naive Bayes model trained
```

Task 5:

Source Code And Output:

```
[16]
✓ 0s
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB

data = load_iris()

X = data.data
y = data.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

model = GaussianNB()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)
```

```
model = GaussianNB()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("Predictions:", y_pred)
print("Actual values:", y_test)

...
Predictions: [1 0 2 1 1 0 1 2 1 1 2 0 0 0 0 1 2 1 1 2 0 2 0 2 2 2 2 2 0 0]
Actual values: [1 0 2 1 1 0 1 2 1 1 2 0 0 0 0 1 2 1 1 2 0 2 0 2 2 2 2 2 0 0]
```

Task 6:

Source Code And Output:

```
7]
Os
```

```python
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

data = load_iris()

X = data.data
y = data.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

model = GaussianNB()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)
```

```
]
Os
```

```python
    X, y, test_size=0.2, random_state=42
)

model = GaussianNB()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
print("Confusion Matrix:")
print(confusion_matrix(y_test, y_pred))
print("Classification Report:")
print(classification_report(y_test, y_pred))
```

```
Accuracy: 1.0
Confusion Matrix:
[[10  0  0]
 [ 0  9  0]
 [ 0  0 11]]
Classification Report:
```

```
Accuracy: 1.0
Confusion Matrix:
[[10  0  0]
 [ 0  9  0]
 [ 0  0 11]]
Classification Report:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00        10
           1       1.00      1.00      1.00         9
           2       1.00      1.00      1.00        11

    accuracy                           1.00        30
   macro avg       1.00      1.00      1.00        30
weighted avg       1.00      1.00      1.00        30
```

Task 7:

Source Code And Output:

```
[18]
✓ 0s

from sklearn.datasets import load_iris
from sklearn.naive_bayes import GaussianNB

data = load_iris()

X = data.data
y = data.target

model = GaussianNB()
model.fit(X, y)

new_data = [[5.1, 3.5, 1.4, 0.2]]

prediction = model.predict(new_data)

print("Predicted Class:", data.target_names[prediction[0]])

Predicted Class: setosa
```